

成员操作手册 V1.2.0

成员操作手册（V1.2.0）

适用对象：使用 CodeBuddy + POM 流程的团队成员 前置条件：方案已实施完成，管理员已完成基础配置 版本：V1.2.0 | 日期：2026-07-21 V1.2.0变更：项目隔离架构（projects/{project}/）+ V5.3.0流程优化（checkpoint 3个+版本沉淀3步+经验分层+知识库三层结构）+ /close自动Git提交

一、角色说明

角色	谁来担任	做什么
管理员	Jojo	维护流程引擎、注册项目+授权成员、审核知识库、管理部署、审核流程改进
技术成员	开发/产品技术	使用 CodeBuddy 命令走完整 POM 流程，开发自己的项目
产品成员	产品/运营/设计	不使用 CodeBuddy，只通过链接查看部署的高保真原型

二、技术成员操作说明

2.1 首次入驻（一次性）

管理员先做

管理员执行一条命令完成注册：
/register-member --project=crm-portal --prefix=CRM --member=zhangsan
→ 自动完成：.access-control.yml + PROJECT-INDEX.yml + 端口分配 + CODEOWNERS + 项目骨架 + 部署配置

成员做

```
# 1. 克隆工作空间
git clone https://gitee.com/jojo180712/pom-workspace.git
cd pom-workspace

# 2. 配置 Git 身份
git config user.name "zhangsan"
git config user.email "zhangsan@example.com"
```

```
# 3. 打开 CodeBuddy, 开始使用
# AI 自动识别你的身份 (通过 git user.name 匹配 .access-control.yml)
```

2.2 多项目管理

一个成员可以同时负责多个项目。每个项目有独立的目录、端口、数据库。

V1.2.0架构变更：项目目录从 projects/{owner}/{project}/ 改为 projects/{project}/，知识属于项目不属于个人，成员可跨项目协作。

创建第二个项目

```
# 场景：张三已有 erp-portal, 现在要新建 crm-portal

# 1. 联系管理员注册新项目+授权
# 管理员执行：/register-member --project=crm-portal --prefix=CRM --member=zhangsan
# 自动分配：新端口（如4002）+ 新数据库名（crm-portal-sim）

# 2. 启动新项目需求流
/init --project=crm-portal --type=新增 --desc=CRM门户v1.0

#    → ensure-project.sh 自动创建骨架：
#    projects/crm-portal/
#    ├── .env.development (VITE_DEV_PORT=4002, VITE_PROJECT_ID=crm-portal)
#    ├── .env.sim (VITE_PROJECT_ID=crm-portal)
#    ├── vite.config.ts (端口读环境变量)
#    └── src/adapters/sim/db.ts (数据库名 crm-portal-sim)

# 3. 新项目独立运行，不影响已有项目
```

项目切换

```
# 成员可以同时开发多个项目，通过 --project 参数切换

# 开发 erp-portal
cd projects/erp-portal
npm run dev # → localhost:4001 (erp-portal)

# 开发 crm-portal (另一个终端窗口)
cd projects/crm-portal
npm run dev # → localhost:4002 (crm-portal)

# 执行命令时通过 --project 指定操作哪个项目
/init --project=erp-portal --type=迭代 --desc=...
/test --project=crm-portal
/close --project=erp-portal
```

多项目数据隔离

项目	开发端口	IndexedDB数据库名	部署路径
erp-portal	4001	erp-portal-sim	/erp-portal/{version}/
crm-portal	4002	crm-portal-sim	/crm-portal/{version}/

- 不同项目的浏览器数据完全隔离（不同的 IndexedDB 数据库）
- 同一项目的不同版本数据自动延续（版本升级不丢数据）
- 部署到 EdgeOne Pages 后通过路径路由区分

2.3 日常开发流程

场景A：新项目从零开始

- 步骤1：学习项目需求（如有现有文档）
/learn-project --project=erp-portal --source=local-dir:./需求文档
→ 产出存入 knowledge/projects/erp-portal/需求文档/
- 步骤2：学习领域知识（可选，贡献到共享知识库）
/learn-domain --category=trade --name=交易通用 --source=http-url:https://...
→ 产出存入 knowledge/domains/trade/（管理员审核后全员可用）
- 步骤3：启动需求流
/init --project=erp-portal --type=新增 --desc=ERP门户v1.0
- 步骤4：走完 POM 流程（V5.3.0：只需3个人工确认卡点）
需求(BA) → [卡点①确认范围] → 设计(UX) → 自动流转 → 架构(Arch) → [卡点②确认方案]
→ 开发(FD) → 测试(QA) → 自动流转 → 验收(AC) → [卡点③确认交付]
- 步骤5：版本冻结 + 自动部署 + 自动Git提交
/close --project=erp-portal
→ 自动版本沉淀（3步精简，close提速50%）
→ 自动构建 + 部署到 EdgeOne Pages + 输出访问链接
→ 自动 git add + commit + push（成员无需手动Git操作）

场景B：迭代已有项目

- /init --project=erp-portal --type=迭代
→ CodeBuddy 自动加载上一版本沉淀 + 项目级经验 + 跨项目经验库
→ 继续 POM 流程... → /close → 自动部署 v2.0.0 + 自动Git提交

场景C：修复 Bug

- /init --project=erp-portal --type=修复 --desc=修复客户列表分页问题
→ 跳过需求/设计/架构，直接 FD → QA → AC
→ /close → 自动部署 v1.0.1 + 自动Git提交

2.4 V5.3.0流程优化说明

Checkpoint 3个卡点（原5个减至3个）

卡点	阶段	需要做什么	原状态
①	需求分析后	确认需求范围无误	保留
②	架构评审后	确认技术方案可行	保留

卡点	阶段	需要做什么	原状态
③	验收完成后	确认交付物达标	保留
-	设计完成后	自动流转到架构（无需确认）	V5.3.0取消
-	测试完成后	自动流转到验收（无需确认）	V5.3.0取消

版本沉淀3步精简（原8步减至3步）

/close后自动执行：
step1: 沉淀准备（目录创建+需求快照+Review合并归档+大脑记忆导出）
step2: 经验分层提炼+追溯链快照
step3: 索引更新+质量校验
→ close提速50%

经验分层（项目级+跨项目级）

/close后经验自动分层：
项目级经验 → knowledge/projects/{project}/经验/
→ 仅本项目可复用，直接写入

跨项目级经验 → knowledge/common/经验库/
→ 通用经验，直接追加，其他项目可参考

流程改进建议 → knowledge/common/经验库/_pending/
→ 待管理员 /review-process-improvements 审核后提升为正式经验

2.5 知识库三层结构（V5.3.0新增）

Layer 1: 全局共享 (common/ + domains/)	
所有项目共享，管理员维护，成员只读	
Layer 2: 项目共享 (projects/{project}/)	
同一项目授权成员共享，成员可读写	
Layer 3: 个人工作过程 (Git commits + branches)	
成员个人开发记录，不进入知识库	

成员可读写： projects/{授权项目}/ 下的所有内容（需求文档、版本沉淀、项目经验） 成员只读：
common/（流程规范+跨项目经验库）、domains/（领域知识）

2.6 Git协作操作

日常工作流

```
# 1. 每天开始工作前：拉取最新代码
git checkout main
git pull origin main
```

```
# 2. 创建工作分支 (按项目+版本命名)
git checkout -b zhangsan/erp-portal-v1

# 3. 开发 (使用 CodeBuddy 命令)
# /init --project=erp-portal --type=新增 --desc=...
# ... 需求→设计→开发→测试→验收

# 4. 提交代码 (只提交自己项目的文件)
git add projects/erp-portal/
git commit -m "feat(erp-portal): v1.0.0 需求完成"

- 客户管理模块
- 统一沟通模块
- AI话术推荐"

# 5. 推送到远端
git push origin zhangsan/erp-portal-v1

# 6. 在 Git 平台创建 Pull Request
#   → 标题: feat(erp-portal): v1.0.0 需求完成
#   → 等待 CODEOWNER 审批
#   → 合并到 main

# 7. 合并后删除工作分支
git checkout main
git pull origin main
git branch -d zhangsan/erp-portal-v1
```

V1.2.0优化： /close 后自动执行 git add + commit + push，成员只需在 Gitee 点合并按钮。

多项目并行开发的 Git 操作

```
# 场景: 同时开发 erp-portal 和 crm-portal

# 为每个项目创建独立分支
git checkout -b zhangsan/erp-portal-v2    # erp-portal 迭代
git checkout main
git checkout -b zhangsan/crm-portal-v1    # crm-portal 新项目

# 分别提交 (只提交对应项目的文件)
# erp-portal 分支
git checkout zhangsan/erp-portal-v2
git add projects/erp-portal/
git commit -m "feat(erp-portal): v2.0.0 迭代"

# crm-portal 分支
git checkout zhangsan/crm-portal-v1
git add projects/crm-portal/
git commit -m "feat(crm-portal): v1.0.0 新项目"

# 分别创建 PR, 分别合并
```

2.7 反馈提交

成员遇到问题或有改进建议时，通过以下方式提交反馈：

方式A：通过 /feedback 命令（流程内反馈）

```
# 在 /close 之前随时可以提交反馈
/feedback --project=erp-portal --msg="需求分析阶段缺少业务规则模板，导致BR编写不规范"
```

方式B：通过 Git Issue（流程外反馈）

在 Git 仓库创建 Issue:

标题: [反馈] ensure-project.sh 端口分配未检测已占用端口

内容:

- 问题描述: ...
- 复现步骤: ...
- 期望行为: ...
- 实际行为: ...
- 截图: ...

标签: feedback / bug / enhancement

方式C：直接告知管理员

直接向管理员描述问题，管理员会:

- 记录到 problem-ledger
- 评估影响范围
- 安排修复优先级
- 通过 /fix 或流程调整解决

V5.3.0新增：成员 /close 后，流程改进建议会自动收集到 common/经验库/_pending/，管理员通过 /review-process-improvements 审核后应用到流程引擎。

2.8 成员能做什么 / 不能做什么

能做

操作	命令
脑暴	/brainstorm --brainstorm-type=regular --project=xxx
启动需求	/init --project=xxx --type=新增 --desc=...
学习项目	/learn-project --project=xxx --source=...
学习领域	/learn-domain --category=... --name=... --source=...
测试	/test --project=xxx
验收	/accept --project=xxx
反馈	/feedback --project=xxx --msg=...
关闭版本	/close --project=xxx
部署	/deploy --project=xxx
报告	/report --project=xxx --report-type=...
交付标注	/handoff --project=xxx

操作	命令
查看部署状态	<code>/deploy --status</code>
不能做	
操作	原因
查看/修改 <code>.codebuddy/configs/</code>	流程引擎，管理员私有
查看/修改 <code>.codebuddy/scripts/</code>	脚本工具，管理员私有
查看/修改 <code>.codebuddy/knowledge/common/</code>	方法论知识库，管理员私有
修改 <code>deploy/</code>	部署配置，管理员私有
修改未授权的项目	只能操作 <code>accessible_projects</code> 列表中的项目
直接编辑共享知识库	只能通过命令贡献，管理员审核后才可见

三、产品成员操作说明

产品成员不使用 CodeBuddy，只查看部署结果。

3.1 访问高保真原型

- 步骤1：打开访问门户 URL（管理员提供）
- 步骤2：输入账号密码登录（EdgeOne 访问保护）
- 步骤3：查看项目版本列表
- 步骤4：点击【访问原型】查看高保真原型（可交互操作）
- 步骤5：点击【查看PRD】查看需求文档

3.2 提反馈

发现问题 → 截图 + 文字描述 → 发给对应技术成员 → 技术成员 `/feedback` 提交

四、常见问题

Q1：成员有多个项目怎么管理？

每个项目有独立的目录、端口、数据库。通过 `--project` 参数切换操作不同项目。可以同时在不同终端窗口开发不同项目。管理员通过 `accessible_projects` 授权你访问哪些项目。

Q2：项目切换时数据会串吗？

不会。每个项目的 IndexedDB 数据库名不同（`{VITE_PROJECT_ID}-sim`），数据完全隔离。

Q3：版本升级后数据还在吗？

在。同一项目不同版本共用同一个 IndexedDB 数据库，版本升级数据自动延续。

Q4：部署失败怎么办？

部署失败不阻断版本冻结。执行 /deploy --project=xxx --retry 手动重试。

Q5：换电脑怎么办？

```
git clone https://gitee.com/jojo180712/pom-workspace.git
cd pom-workspace
git config user.name "zhangsan"
git config user.email "zhangsan@example.com"
cd projects/erp-portal && npm install
# 代码在 Git 中恢复, IndexedDB 数据新电脑重新初始化
```

Q6：/close后还需要手动提交Git吗？

不需要。V1.2.0起 /close 后自动执行 git add + commit + push，你只需在 Gitee 上点合并按钮。

Q7：我的流程改进建议怎么被采纳？

你 /close 后，涉及流程/Agent/Skill改进的建议会自动收集到 common/经验库/_pending/。管理员定期执行 /review-process-improvements 审核，通过后会生成流程升级方案并应用到流程引擎。

Q8：设计阶段和测试阶段为什么不需要确认了？

V5.3.0优化后，设计完成后自动流转 to 架构评审，测试完成后自动流转 to 验收。只需在需求确认(①)、架构确认(②)、验收确认(③)三个关键卡点操作，减少40%的用户交互。

五、成员速查卡

常用命令

场景	命令
启动开发服务器	cd projects/{project} && npm run dev
构建仿真原型	npm run build:sim
启动新需求	/init --project=xxx --type=新增 --desc=...
迭代版本	/init --project=xxx --type=迭代
修复Bug	/init --project=xxx --type=修复 --desc=...
执行测试	/test --project=xxx
执行验收	/accept --project=xxx
提交反馈	/feedback --project=xxx --msg=...
关闭版本（自动部署+自动Git）	/close --project=xxx

场景	命令
手动部署	/deploy --project=xxx
查看部署状态	/deploy --status
查看报告	/report --project=xxx --report-type=...

POM流程卡点 (V5.3.0: 3个)

卡点	阶段	操作
①	需求分析后	确认范围
②	架构评审后	确认方案
③	验收完成后	确认交付

Git 速查

场景	命令
拉取最新	git pull origin main
创建分支	git checkout -b {member}/{project}-v{version}
提交	git add projects/{project}/ && git commit -m "..."
推送	git push origin {branch}
合并后清理	git checkout main && git pull && git branch -d {branch}